

COS30018 – OPTION C

TASK 6 REPORT

Student Name: Syed Muhammad Mahdi

Student ID: 105172195

Date: 28 September 2025

1. Introduction

Task 6 continues the efforts done in Task 5 to scale the machine learning models of stock prices prediction. The primary goal was to test the various sequence-to-sequence design alternatives: univariate multi-step, multivariate single-step and multi-variate multi-step forecasting. This enabled us to test the performance of models including LSTM, GRU and RNN with different input-output configurations.

2. Environment Setup

The Python with TensorFlow/Keras, pandas, NumPy, Matplotlib, and yfinance were used to conduct the experiments based on the price data of stocks retrieved using yfinance. The Task 5 base code was built upon and extended with new windowing functions to allow multi-step and multi-variable forecasting.

3. Implementation

There were three implementation methods on sequence generation:

- make_windows_multistep_univariate - to make predictions of the subsequent K closing prices with past Close values alone.
- make_windows_multivariate_single_step - to predict the next Close with the help of OHLCV.
- make_windows_multivariate_multistep – to forecast the next K Closes with OHLCV features.

All models with LSTM, GRU, and RNN layers with an equal number of layers (1 2), with 64 and 128 hidden units, and dropout regularization were searched. EarlyStopping, ReduceLROnPlateau and ModelCheckpoint callbacks were used in training.

4. Results

The experiments were executed in multi-single mode and OHLCV characteristics predicted the succeeding Close. The best settings by Mean Absolute Error (MAE) are as below:

The screenshot shows a VS Code editor with a Python script `task6_run.py` and its execution results in the terminal. The script defines a `main` function that iterates over a grid of hyperparameters (layer_type, num_layers, hidden_size, dropout, lr, batch_size, epochs) and runs a multi-single forecasting model. The terminal output shows the progress of the search, including training and validation loss, MAE, and mape for each configuration. The results are saved to `results/task6_multi_single_results.csv`.

```
task6_run.py > ...
254 def main():
255     for layer_type, num_layers, hidden_size, dropout, lr, batch_size, epochs in product(
256         SEARCH_SPACE["layer_type"], SEARCH_SPACE["num_layers"], SEARCH_SPACE["hidden_size"],
257         SEARCH_SPACE["dropout"], SEARCH_SPACE["lr"], SEARCH_SPACE["batch_size"], SEARCH_SPACE["epochs"]
258     ):
259         run_name = f"{MODE}_{layer_type}_{num_layers}_{hidden_size}_{dropout}_{batch_size}"

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

41/41 3s 34ms/step - loss: 0.0019 - mae: 0.0344 - mape: 3798.5000 - val_loss: 0.0013 - val_mae: 0.0286 - v
al_mape: 4.0664 - learning_rate: 0.0010
Epoch 8/40
41/41 1s 34ms/step - loss: 0.0013 - mae: 0.0277 - mape: 1837.5339 - val_loss: 7.9129e-04 - val_mae: 0.0212
- val_mape: 3.0041 - learning_rate: 0.0010
Epoch 9/40
41/41 2s 36ms/step - loss: 0.0016 - mae: 0.0314 - mape: 81429.8750 - val_loss: 5.8864e-04 - val_mae: 0.017
6 - val_mape: 2.5094 - learning_rate: 5.0000e-04
Epoch 10/40
41/41 1s 33ms/step - loss: 0.0011 - mae: 0.0259 - mape: 23646.9414 - val_loss: 6.5563e-04 - val_mae: 0.018
7 - val_mape: 2.6522 - learning_rate: 5.0000e-04
Epoch 11/40
41/41 1s 33ms/step - loss: 0.0014 - mae: 0.0294 - mape: 9016.4785 - val_loss: 0.0011 - val_mae: 0.0262 - v
al_mape: 3.7226 - learning_rate: 5.0000e-04
Epoch 12/40
41/41 1s 35ms/step - loss: 9.9645e-04 - mae: 0.0247 - mape: 39551.1211 - val_loss: 6.7699e-04 - val_mae: 0
.0191 - val_mape: 2.7062 - learning_rate: 5.0000e-04

Saved results to: results/task6_multi_single_results.csv
Top configs by MAE:
mode run layer_type num_layers hidden_size mae loss mape
5 multi_single multi_single_GRU_L1_H128_D0.2_B32 GRU 1 128 0.017597 0.000534 2.126627
7 multi_single multi_single_GRU_L2_H128_D0.2_B32 GRU 2 128 0.020169 0.000733 2.445862
4 multi_single multi_single_GRU_L1_H64_D0.2_B32 GRU 1 64 0.020605 0.000676 2.467803
9 multi_single multi_single_RNN_L1_H128_D0.2_B32 RNN 1 128 0.023203 0.000841 2.764725
1 multi_single multi_single_LSTM_L1_H128_D0.2_B32 LSTM 1 128 0.023385 0.000962 2.831943
(base) PS D:\t4>
```

5. Observations

GRU average performancescustomer-distributed: GRU dances around the lowest MAE of all tested models (0.0176).

- Adding layers increased performance in certain situations, however, it might cause greater validation loss because of overfitting.

RNN and LSTM models did quite well but still failed in comparison to GRU in this configuration.

Multi-single step method used successfully applied the characteristics of OHLCV in ensuring consistency and correct prediction.

6. Conclusion

Task 6 was an effective extension of Task 5 because it used more advanced forecasting mechanisms based on sequence-to-sequence models. GRU was found to be the most suitable architecture in terms of multi-single forecasting of Apple stock prices. The results show that model choice and windowing choices are significant in tasks of forecasting time-series.